

DAYTRACE

Product Requirements Document

Android Offline-First Task Planner, Activity Logger and Daily Report System

Version	1.0
Status	Approved for MVP Implementation
Product Owner	Zain UI Haq
Platform	Android / Flutter
Prepared	29 July 2026

How to Use This Document

This PRD defines what DayTrace must do and when the MVP is considered complete. The accompanying Markdown, SQL, architecture, and AI-agent files are designed for direct use inside an AI coding IDE. The PDF is the readable reference; the repository files are the implementation context.

Table of Contents

1. 1. Document Control
2. 2. Executive Summary
3. 3. Problem Statement
4. 4. Product Vision and Goals
5. 5. Non-Goals for Version 1
6. 6. Target Users and Personas
7. 7. Core Product Principles
8. 8. Information Architecture
9. 9. Detailed Functional Requirements
10. 10. User Stories and Acceptance Criteria
11. 11. Notification Rules
12. 12. Data Model and Business Rules
13. 13. Technical Architecture
14. 14. AI-Assisted Development Rules
15. 15. Non-Functional Requirements
16. 16. Analytics and Logging
17. 17. Backup, Restore, and Export
18. 18. Edge Cases
19. 19. Testing Strategy
20. 20. Implementation Phases
21. 21. Definition of Done for MVP
22. 22. Future Roadmap

1. Document Control

Product: DayTrace

Document: Product Requirements Document (PRD)

Version: 1.0

Status: Approved for MVP implementation

Platform: Android mobile application

Primary technology: Flutter

Product owner: Zain Ul Haq

Primary implementation method: AI-assisted development through an IDE

This document is the source of truth for product behavior, scope, terminology, priorities, acceptance criteria, and implementation boundaries. When another instruction conflicts with this PRD, the most recently approved written decision must win and the change must be recorded in CHANGELOG.md.

2. Executive Summary

DayTrace is an Android productivity application for people whose work changes throughout the day and who struggle to remember what they did, when they did it, what remains pending, and what must happen next.

The application combines five connected capabilities:

1. Quick task capture and scheduling.
2. Real-time activity tracking with start, pause, resume, and complete actions.
3. A chronological daily timeline that exposes tracked and untracked time.
4. Reliable local reminders for planned work and missing activity logs.
5. Automatic daily and weekly reports, including an optional AI-written summary.

DayTrace is not a generic checklist. Its defining promise is: "Plan the work, record the reality, and explain the day."

The first release must work offline, require no login, store data locally, deliver local notifications, and remain usable during unreliable internet or power conditions. Cloud backup and multi-device synchronization are future capabilities; the data model and repository boundaries must allow them later without rewriting the entire app.

3. Problem Statement

The target user does not follow a predictable work plan. Tasks arrive from managers, machines, coworkers, home responsibilities, and studies. Activity duration is unknown in advance. By the end of the day the user may forget completed work, lose pending tasks, miss study sessions, or be unable to answer a manager who asks for a time-based activity report.

Existing calendar apps are good at events but weak at recording unscheduled real work. Traditional to-do apps record completion but usually do not preserve an accurate timeline. Time trackers often assume organized projects and deliberate timer usage. DayTrace connects these behaviors while keeping the capture flow fast enough for a factory floor, office, field job, study desk, or home.

4. Product Vision and Goals

Vision

Create a personal operating log that reduces dependence on memory and turns a chaotic day into a reliable, searchable record.

MVP goals

- Allow a new task or activity to be recorded in under 10 seconds.
- Record actual start, pause, resume, and end times accurately.
- Show what the user is doing now and what should happen next.
- Generate a trustworthy daily timeline and end-of-day summary.
- Notify the user about scheduled tasks and prolonged untracked time.

- Support personal, study, home, and company contexts without creating separate apps.
- Preserve all core functions without internet access or account creation.
- Make the codebase understandable to AI coding agents through explicit architecture and rules.

Success indicators for the product owner during the first 30-day dogfood period

- At least 20 active usage days.
- At least 70 percent of working-hour time is categorized or intentionally marked untracked.
- At least 80 percent of scheduled reminders are either completed, rescheduled, or explicitly dismissed.
- A usable daily report can be produced in less than 30 seconds.
- No loss of local task or time-entry data after normal app restarts or device reboots.

5. Non-Goals for Version 1

Version 1 will not include:

- Team accounts, supervisor dashboards, attendance, payroll, or employee surveillance.
- Full ERP, inventory, manufacturing production, CRM, or project billing.
- Web, iOS, desktop, or smartwatch applications.
- Real-time collaboration, chat, comments, or shared task assignment.
- Mandatory login or internet dependency.
- Automatic background microphone recording or continuous location tracking.
- Fully automatic activity recognition from phone usage.
- Complex Google Calendar two-way synchronization.
- Subscription payments, advertisements, or public marketplace release requirements.

These exclusions protect the core use case. Features may be reconsidered only after stable personal use confirms that the core workflow works.

6. Target Users and Personas

Primary persona: Variable-duty employee

A factory, office, operations, maintenance, or field employee whose tasks change frequently and who may need to explain completed work to a senior.

Secondary persona: Working student

A person balancing employment, university, courses, assignments, and personal responsibilities. They need reminders and actual study-time records rather than optimistic plans.

Secondary persona: Freelancer or solo worker

A person who needs a lightweight activity log and daily report without configuring clients, projects, invoices, or enterprise systems.

Initial dogfood user

Zain, based in Karachi, works in a mechanical manufacturing environment and studies software engineering. He uses Android and needs fast operation, offline reliability, understandable reports, and minimal manual administration.

7. Core Product Principles

23. 1. Capture first, organize later: a user must be able to start work immediately with only a title.
24. 2. Reality beats planning: estimated time is useful, but actual time is the authoritative record.
25. 3. One active timer: only one activity may actively accumulate time at once.
26. 4. Offline is normal: network availability must never block task or timer operations.
27. 5. Notifications require action: reminder notifications should offer useful actions such as Start, Snooze, Complete, or Skip.
28. 6. No silent data loss: destructive actions require confirmation or undo, and all state transitions must be persisted immediately.
29. 7. Explainable automation: smart prompts must be configurable, dismissible, and never pretend to know what the user did.

30. 8. Low cognitive load: the Today screen must answer "What am I doing?", "What is next?", and "What did I do?" without navigation gymnastics.

8. Information Architecture

Primary bottom navigation

1. Today
2. Tasks
3. Timeline
4. Reports
5. Settings

Global quick action

A floating action button or prominent action opens Quick Add. It must support Save Task and Start Now.

Primary entities

- Task: planned or actionable work.
- Time Entry: a continuous interval spent on a task or general activity.
- Category: context such as Company, Study, Personal, Home, Break, or Meeting.
- Reminder: a scheduled notification connected to a task or routine.
- Daily Note: optional text attached to a date.
- Generated Summary: deterministic or AI-generated report text saved for later reuse.

9. Detailed Functional Requirements

31. 9.1 Onboarding

- First launch must not require an account.
- Show no more than three concise onboarding pages: Capture, Track, Review.
- Ask for notification permission only when the value is explained.
- Ask for microphone permission only when the user first selects voice entry.
- Collect optional working days, working start/end time, and untracked-time prompt interval.
- Create default categories: Company, Study, Personal, Home, Meeting, Break.

32. 9.2 Today screen

- Display date and greeting.
- Display the current active activity, elapsed duration, start time, and Pause/Resume/Complete controls.
- If no activity is active, show Start Activity and suggested next tasks.
- Show today's planned tasks grouped as Overdue, Due Now, Later, and Completed.
- Show summary cards for tracked time, completed tasks, pending tasks, and untracked gaps.
- Provide a direct link to the daily timeline and end-of-day review.

9.3 Quick Add

Required minimum: title.

Optional fields: description, category, priority, planned date, due time, reminder, estimated duration, recurrence, subtasks, tags, and notes.

Actions: Save Task, Save and Start, Cancel.

Defaults should use the last selected category and current date where sensible.

9.4 Task lifecycle

Statuses: planned, in_progress, paused, completed, cancelled, archived.

- Starting a task creates a new open time entry.
- Starting a second task while another is active must present: Pause current and start new; Complete current and start new; Cancel.
- Pausing closes the current interval and marks the task paused.
- Resuming creates a new interval; it does not edit the previous interval.

- Completing closes any active interval and records completed_at.
- A completed task can be reopened; the event must be visible in audit metadata.
- Cancelled and archived tasks must not appear in default active lists.

33. 9.5 Timer and time entries

- Timer display is derived from persisted timestamps, not a fragile in-memory counter.
- Only one time entry can have a null end time.
- Timer state must survive app backgrounding, process death, and device restart.
- Manual time entry creation and editing are allowed.
- The user may split, merge, or reassign time entries with confirmation.
- Duration is calculated from start and end timestamps; stored duration may be cached but must be recalculable.
- Entries crossing midnight must be represented correctly in daily reports.

34. 9.6 Daily timeline

- Display all time entries chronologically.
- Display gaps between entries during configured tracking hours.
- A gap can be assigned to a task/category, marked Break, marked Untracked, or ignored.
- Timeline editing must prevent overlaps unless the user explicitly resolves them.
- Filters: category, task, tracked/untracked, and date.

35. 9.7 Task planning and reminders

- Support one-time reminders and basic recurrence: daily, selected weekdays, weekly, monthly.
- Notification actions: Start, Complete, Snooze, Dismiss.
- Snooze presets: 10 minutes, 30 minutes, 1 hour, custom.
- Overdue tasks remain visible until completed, rescheduled, cancelled, or archived.
- Rescheduling must update notification scheduling atomically.
- Local notifications must be restored after device reboot where Android allows it.

36. 9.8 Smart untracked-time prompt

- During configured working/tracking hours, if no task is active for the configured interval, send a notification: "What are you doing?"
- Default interval: 60 minutes; options: off, 30, 45, 60, 90, 120 minutes.
- Actions: Start a task, Add past activity, Break, Meeting, Ignore.
- Ignoring a prompt must not fabricate a time entry.
- Prompts must respect quiet hours and should not repeat more frequently than configured.

37. 9.9 Voice entry

- From Quick Add or Today, the user can tap a microphone and dictate a task title or command.
- MVP voice parsing supports simple patterns such as: "Start machine inspection", "Remind me to study at 8 PM", and "Add production report for tomorrow".
- If parsing confidence is low, populate editable fields and require user confirmation.
- Voice entry must not listen continuously.
- Core app remains fully usable without microphone permission.

9.10 Reports

Daily report must include:

- Total tracked duration.
- Duration by category.
- Completed, pending, overdue, and cancelled task counts.
- Chronological activity list with start/end/duration.
- Untracked duration during configured tracking hours.
- Optional notes and saved summary.

Weekly report must include:

- Tracked duration by day and category.

- Completion count and completion rate.
- Most time-consuming tasks.
- Overdue carry-forward tasks.
- Average daily tracked duration.

Exports in MVP:

- Copy report as plain text.
- Share through Android share sheet.
- Generate a simple PDF report.

38. 9.11 AI summary

- The app must always provide a deterministic non-AI summary generated locally from structured data.
- Optional AI summary converts the structured report into concise professional language.
- AI use must be explicit; never upload data silently.
- Before sending, show or state what data will be transmitted.
- Do not embed a permanent secret API key in the released mobile application.
- MVP development may support a user-provided API key stored using secure storage, or a developer-configured proxy. Public release should use a server-side proxy with authentication, rate limits, and cost controls.
- If AI fails or internet is unavailable, fall back to the local summary.
- Save generated text with provider/model metadata and source date range.

39. 9.12 Search

- Search tasks and time-entry notes by title, description, category, and date.
- Filters: status, category, priority, planned date, completion date.
- Recent searches are optional; no cloud search in MVP.

40. 9.13 Settings

- Working/tracking days and hours.
- Smart prompt interval and quiet hours.
- Reminder defaults and snooze presets.
- Category management.
- Theme: system, light, dark.
- Week start day.
- Time format: 12-hour or 24-hour.
- Export all data.
- Import/restore backup.
- Clear data with typed confirmation.
- Optional AI configuration and privacy explanation.

10. User Stories and Acceptance Criteria

US-01 Quick activity start

As a user, I can start an unplanned activity quickly so that work is recorded before I forget.

Acceptance:

- From Today, starting an activity requires only a title and at most two taps after text entry.
- A persisted open time entry exists immediately.
- The active timer remains correct after closing and reopening the app.

US-02 Switch activity

As a user, I can safely switch tasks without overlapping timers.

Acceptance:

- The application detects an active task.
- It asks whether to pause or complete it.
- Exactly one open time entry remains after the switch.

US-03 Schedule study

As a user, I can schedule a study task and receive an actionable reminder.

Acceptance:

- A notification appears near the selected time.
- Start opens or starts the correct task.
- Snooze reschedules it.
- Completing from the notification updates the task and Today screen.

US-04 Explain the day

As a user, I can show my senior what I did and when.

Acceptance:

- The daily report contains ordered entries with start, end, and duration.
- Manual corrections are reflected immediately.
- Report can be copied and shared.

US-05 Recover missing time

As a user, I can categorize a gap after the fact.

Acceptance:

- Timeline identifies a gap inside tracking hours.
- User can add a past activity for all or part of the gap.
- Overlap validation prevents corrupt entries.

US-06 Voice capture

As a user with busy hands, I can dictate a new task.

Acceptance:

- Speech result is shown for review.
- No task is silently created from uncertain parsing.
- Denied microphone permission does not block manual entry.

US-07 AI report fallback

As a user, I can still get a report when AI is unavailable.

Acceptance:

- Local summary is always available.
- AI failure shows a clear message without losing report data.
- Retry does not duplicate saved summaries.

US-08 Data safety

As a user, my records survive normal interruptions.

Acceptance:

- Data persists after app restart and Android process termination.
- An active timer is reconstructed from timestamps.
- Backup export produces a readable file and restore validates its schema version.

11. Notification Rules

Notification channels

- Task reminders: high importance, user configurable sound/vibration.
- Active timer: optional ongoing low-importance notification with Pause and Complete actions.
- Smart prompts: default importance, no aggressive repeated sound.
- End-of-day review: default importance.

Rules

- Every scheduled notification must have a stable identifier.
- Editing, completing, cancelling, or deleting a task must cancel obsolete notifications.
- Notification actions must use idempotent handlers.
- Exact alarms must not be assumed available on every Android version or device. Use appropriate scheduling APIs and explain battery optimization limitations in Settings.
- The app must request only permissions needed for selected features.

12. Data Model and Business Rules

Core tables

- categories
- tasks
- subtasks
- time_entries
- reminders
- recurrence_rules
- daily_notes
- generated_summaries
- app_settings
- schema_migrations

Important invariants

- IDs are UUID strings generated locally to support later synchronization.
- All persisted timestamps use UTC ISO-8601 or integer epoch milliseconds; UI converts to local time.
- Exactly one time entry may be open globally.
- Task status and open time entry must remain consistent within one database transaction.
- Time entries must satisfy end_at > start_at when end_at is present.
- Time entries may not overlap by default.
- Deletion should use soft-delete fields for syncable records where practical.
- Each record includes created_at and updated_at.
- Future cloud synchronization fields may include sync_status, server_version, and deleted_at, but they must not complicate the initial UI.

The canonical SQL-like schema is included in DATABASE_SCHEMA.sql. Flutter implementation should use Drift migrations and typed data access rather than scattered raw SQL.

13. Technical Architecture

Recommended architecture

- Flutter Android application.
- Feature-first Clean Architecture with pragmatic boundaries.
- Presentation: Flutter widgets and Riverpod controllers/providers.
- Domain: entities, value objects, repository contracts, and use cases for state-changing workflows.
- Data: Drift local database, repository implementations, notification service, voice service, AI summary service, export service.

Suggested package structure

```
lib/
  app/
  core/
  features/onboarding/
  features/today/
  features/tasks/
  features/timer/
  features/timeline/
  features/reminders/
  features/reports/
  features/settings/
  shared/
```

Required engineering patterns

- Repository interfaces isolate UI from storage.
- All timer transitions run through one TimerCoordinator/use case.
- Database writes affecting task and timer state use transactions.
- Services are injected; do not call plugins directly from widgets.
- Riverpod providers expose immutable UI state.

- Errors use typed failures or sealed result types rather than swallowed exceptions.
- Navigation uses a declarative router.
- Platform-specific code is isolated behind interfaces.

Recommended libraries, subject to compatibility verification during implementation

- flutter_riverpod / riverpod_annotation
- drift / drift_flutter
- go_router
- flutter_local_notifications
- timezone
- workmanager where justified
- speech_to_text
- permission_handler
- share_plus
- pdf and printing
- flutter_secure_storage for optional secrets
- freezed and json_serializable where they reduce boilerplate
- uuid
- intl

Do not pin package versions in this PRD. The implementation agent must select mutually compatible stable versions and record them in pubspec.yaml and DECISIONS.md.

14. AI-Assisted Development Rules

The implementation will be performed mainly by AI coding agents. Therefore:

- AGENTS.md, PRD.md, ARCHITECTURE.md, DATABASE_SCHEMA.sql, and IMPLEMENTATION_PLAN.md must remain in the repository root or /docs and be read before code changes.
- The agent must inspect existing code before proposing changes.
- Work in small vertical slices that compile and are testable.
- Do not generate placeholder screens presented as completed functionality.
- Do not change architecture, database semantics, or scope without updating DECISIONS.md.
- After every feature, run formatting, static analysis, relevant tests, and a debug build.
- Never expose API keys, OAuth secrets, signing keys, or service-role keys in source control.
- Avoid destructive migrations.
- The agent must state changed files, tests run, remaining risks, and the next recommended task.
- When uncertain, choose the simplest solution satisfying the acceptance criteria and document the assumption.

15. Non-Functional Requirements

Performance

- Cold launch target under 3 seconds on a modest Android device after initial database setup.
- Today screen should become interactive promptly and avoid unnecessary full-table queries.
- Lists should remain smooth with at least 10,000 time entries.

Reliability

- All timer state changes persisted synchronously before UI success is shown.
- Database migrations tested from every released schema version.
- Notification rescheduling is recoverable after reboot and app update.

Accessibility

- Touch targets at least 48 logical pixels where practical.
- Text scales without clipping.
- Controls have semantic labels.
- Color is not the only indicator of status.

Privacy

- Local-first by default.
- Microphone used only after explicit action.
- AI transmission opt-in and transparent.
- Exported backups may contain sensitive work history; warn the user before sharing.

Security

- No secrets committed to Git.
- Optional API keys stored in Android secure storage.
- Imported backups validated before database mutation.
- SQL parameters bound safely through Drift.

Maintainability

- Business rules covered by unit tests.
- Major services replaceable through interfaces.
- No giant god classes or screens with embedded database and plugin logic.

16. Analytics and Logging

MVP does not require third-party analytics.

Use local diagnostic logging in debug builds for:

- Database migrations.
- Timer state transitions.
- Notification scheduling and action handling.
- Backup/restore operations.
- AI request success/failure without logging private content or secrets.

Production logs must avoid task text unless the user explicitly exports diagnostics. Crash reporting may be added later with a privacy disclosure.

17. Backup, Restore, and Export

Backup format

- JSON package with schema_version, exported_at, categories, tasks, subtasks, time_entries, reminders, recurrence rules, daily notes, summaries, and selected settings.
- Optional compressed archive may be added later.

Restore rules

- Validate file structure and schema version before changes.
- Create a safety backup before replacing existing data.
- Initial MVP may support Replace All; Merge is deferred unless implemented safely.
- Report invalid records clearly.

Report PDF export

- Generate on device.
- Include date range, summary metrics, category totals, and chronological entries.
- Avoid including hidden/internal identifiers.

18. Edge Cases

- User starts a task at 11:50 PM and completes it after midnight.
- Device restarts while a timer is running.
- Timezone or system clock changes while a timer is active.
- User edits an entry creating an overlap.
- Reminder fires for a task already completed on another screen.
- Notification permission denied.

- Exact alarm permission unavailable.
- Microphone permission denied or speech service unavailable.
- AI request times out, returns invalid content, or is rate limited.
- Database migration interrupted.
- Backup from a newer unsupported app version.
- Recurring task occurrence skipped or completed late.
- Daylight-saving transitions, despite initial user being in a non-DST locale.
- User force-stops the app, which may prevent scheduled background work until next launch.

Each edge case must result in safe, explainable behavior rather than silent corruption.

19. Testing Strategy

Unit tests

- Timer start/pause/resume/complete transitions.
- One-active-entry invariant.
- Duration calculations and midnight splitting/reporting.
- Recurrence calculation.
- Gap detection.
- Deterministic summary generation.
- Backup validation.

Database tests

- CRUD and transactions.
- Constraints and indexes.
- Migration tests.
- Overlap queries.

Widget tests

- Today states: no active task, active task, overdue tasks, permission warning.
- Quick Add validation.
- Timeline gap actions.
- Report rendering.

Integration tests

- Create task -> schedule reminder -> start -> pause -> resume -> complete -> report.
- Process restart during active timer.
- Notification action idempotency where testable.
- Backup and restore round trip.

Manual Android QA

- At least one low/mid-range physical device.
- Backgrounding, force closing, rebooting, permission denial, battery saver, and offline use.

20. Implementation Phases

Phase 0: Foundation

Project creation, linting, themes, routing, Drift setup, dependency injection, error handling, CI checks, and documentation.

Phase 1: Core task and timer vertical slice

Categories, tasks, quick add, one active timer, Today screen, persistence, and tests.

Phase 2: Planning and reminders

Due dates, recurrence, local notifications, actions, permission UX, and reboot restoration.

Phase 3: Timeline and gap recovery

Chronological timeline, manual entries, overlap protection, gap detection, and smart prompts.

Phase 4: Reports and export

Daily/weekly reports, deterministic summary, share text, and PDF export.

Phase 5: Voice and AI summary

Speech capture with confirmation, optional AI configuration, secure key handling or proxy interface, failure fallback.

Phase 6: Hardening and dogfood release

Backup/restore, migration tests, accessibility, performance, crash fixes, signed internal APK, and 30-day use.

Detailed tasks and gates are in IMPLEMENTATION_PLAN.md.

21. Definition of Done for MVP

The MVP is complete only when:

- All required screens and flows work on a physical Android device.
- Core use is possible without login and internet.
- Timer survives app restarts and accurately records intervals.
- Task reminders and smart prompts are configurable and actionable.
- Daily timeline exposes gaps and permits safe correction.
- Daily and weekly reports work, including local summary and PDF/text sharing.
- Voice entry works with review and graceful permission failure.
- Optional AI summary has privacy disclosure, safe secret handling, and local fallback.
- Backup/restore is tested.
- Static analysis passes with no unresolved serious warnings.
- Critical business logic has automated tests.
- No secrets or debug credentials are committed.
- Documentation matches implementation.

22. Future Roadmap

Candidates after successful dogfooding:

- Supabase account, encrypted cloud backup, and multi-device synchronization.
- Google Calendar import or controlled synchronization.
- Home-screen widgets and quick settings tile.
- Natural-language task parsing improvements.
- Templates and routines.
- Public onboarding, privacy policy, feedback, analytics, and Play Store release.
- Team or manager reports only if validated without turning DayTrace into surveillance software.